

JAVA PROGRAMMING LABORATORY

Experiment No.	06
Date	23/09/26
Title	Implementation of LinkedHashMap Methods in Java

Objective / Aim

To write and execute a Java program that demonstrates commonly used methods of the LinkedHashMap class, including put(), get(), containsKey(), keySet(), values(), entrySet(), and remove(), while observing that LinkedHashMap maintains insertion order.

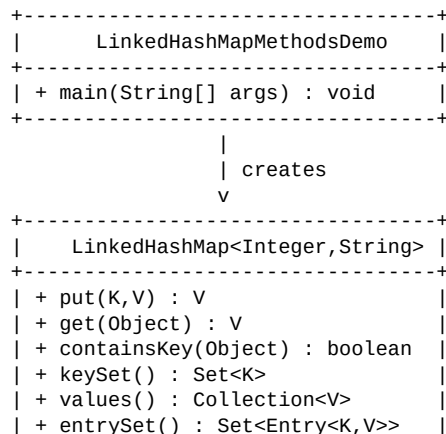
System Requirements

Hardware Requirements	Software Requirements
Computer/Laptop	Operating System: Windows/Linux/macOS
Processor: Intel Core i3 or above	JDK 17 or JDK 21
RAM: 4 GB or above	Java compiler (javac)
Storage: At least 1 GB free space	IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code

Algorithm / Logic Description

1. Import the LinkedHashMap class from the java.util package.
2. Create a LinkedHashMap named employees.
3. Insert four employee ID and name pairs using put().
4. Display the LinkedHashMap after insertion.
5. Use get() to retrieve the employee associated with key 102.
6. Use containsKey() to check whether key 103 exists.
7. Use keySet() to display all keys.
8. Use values() to display all employee names.
9. Use entrySet() to display all key-value pairs.
10. Use remove(103) to remove the employee with ID 103.
11. Display the LinkedHashMap after removal.

UML Class Diagram



```
| + remove(Object) : V |  
+-----+  
|
```

Source Code

```
import java.util.LinkedHashMap;  
  
class LinkedHashMapMethodsDemo {  
  
    public static void main(String[] args) {  
  
        // Creating a LinkedHashMap  
        LinkedHashMap<Integer, String> employees = new LinkedHashMap<>();  
  
        // put(K key, V value)  
        employees.put(101, "Rahul");  
        employees.put(102, "Priya");  
        employees.put(103, "Arjun");  
        employees.put(104, "Sneha");  
  
        System.out.println("After put(): " + employees);  
  
        // get(Object key)  
        System.out.println("Employee with ID 102: "  
            + employees.get(102));  
  
        // containsKey(Object key)  
        System.out.println("Contains key 103? "  
            + employees.containsKey(103));  
  
        // keySet()  
        System.out.println("Keys: " + employees.keySet());  
  
        // values()  
        System.out.println("Values: " + employees.values());  
  
        // entrySet()  
        System.out.println("Key-Value pairs: "  
            + employees.entrySet());  
  
        // remove(Object key)  
        employees.remove(103);  
        System.out.println("After remove(103): "  
            + employees);  
    }  
}
```

Completion and Execution Commands

```
Save as:  
LinkedHashMapMethodsDemo.java  
  
Compile:  
javac LinkedHashMapMethodsDemo.java  
  
Execute:  
java LinkedHashMapMethodsDemo
```

Output

```
After put(): {101=Rahul, 102=Priya, 103=Arjun, 104=Sneha}  
Employee with ID 102: Priya  
Contains key 103? true  
Keys: [101, 102, 103, 104]  
Values: [Rahul, Priya, Arjun, Sneha]  
Key-Value pairs: [101=Rahul, 102=Priya, 103=Arjun, 104=Sneha]  
After remove(103): {101=Rahul, 102=Priya, 104=Sneha}
```

Note: LinkedHashMap maintains the insertion order of entries when they are iterated or displayed. This distinguishes it from HashMap, which does not guarantee iteration order.